Dynamic Spatial Stabilization and Trajectory Tracking of a Spherical Agent on a 6-DOF Robotic Platform

Phillipp Gery (pgery), Kevin Biju Mathew (kb)

***Abstract*—This report presents the implementation of a real-time discrete-time Linear Quadratic Regulator (LQR) controller for stabilizing a marble on the end-effector plate of a UR5e 6-DOF robotic arm in simulation. The simulation environment is built on Gazebo Classic, ROS2 Humble, and MoveIt2 Servo. An 8-dimensional state-space model captures the coupled ball-plate dynamics and the robot's first-order (PT1) actuator lag. The LQR gain is computed offline via the Discrete Algebraic Riccati Equation and applied at 30 Hz. Preliminary results demonstrate successful marble stabilization and autonomous recovery after plate departure. The system is further extended with a priority multiplexer for manual override, marble Lissajous trajectory tracking, and simultaneous TCP Lissajous motion with feedforward tilt compensation.**

***Index Terms*—ROS 2, Gazebo, LQR, MoveIt Servo, UR5e, Trajectory Tracking, Ball-Plate System.**

I. Introduction

PRECISION stabilization of unconstrained objects is a fundamental challenge in robotics. This project implements real-time feedback control of a UR5e 6-DOF robotic arm that balances a marble on a flat plate rigidly attached to the robot's end-effector. The arm tilts the plate via small angular velocity commands, keeping the marble centered. The full system operates in simulation using **Gazebo Classic**, **ROS2 Humble**, and **MoveIt2 Servo**.

The problem couples two dynamical systems: the rolling dynamics of the marble on the plate, and the servo dynamics of the arm responding to velocity commands. Marble acceleration depends on plate tilt angle, while the arm's response to commands is subject to inherent actuator lag. A classical discrete-time **Linear Quadratic Regulator (LQR)** serves as the primary controller, designed around a physics model that explicitly captures both the ball-plate coupling and the robot's first-order (PT1) delay.

Unlike traditional "Labyrinth" setups where the board orientation is directly actuated, this project employs *active surface control*: a 6-DOF arm adjusts end-effector orientation in real time while simultaneously being capable of tracing arbitrary 3D paths with its TCP. This report covers the simulation setup and LQR classical controller constituting Milestone 1, followed by system extensions developed subsequently.

College of Engineering, Purdue University, Indiana, USA.

II. Methods

A. Simulation Environment

1) Robot and World: The UR5e arm is loaded into Gazebo Classic via a URDF xacro that uses the standard `ur_description` macros. A $0.70\,\mathrm{m} \times 0.70\,\mathrm{m} \times 5\,\mathrm{mm}$ plate (mass $0.15\,\mathrm{kg}$) is rigidly attached to the robot's `tool0` link. A virtual frame `plate_tcp` sits at the plate's top surface and serves as the MoveIt Servo control reference frame.

The robot home configuration is defined in the SRDF as joint angles $[0, -\pi/2, \pi/2, -\pi/2, -\pi/2, 0]$, which guarantees zero roll and zero pitch on the plate at startup. The `go_to_pose` node computes the corresponding inverse kinematics solution via MoveIt and commands the arm to this home position, which serves as the initial pose for placing the marble as well as the recovery position the robot returns to after each failure.

2) Marble: The marble is a $15\,\mathrm{mm}$ radius, $50\,\mathrm{g}$ sphere defined in SDF format. Gazebo's ODE physics engine simulates rolling contact on the plate with a friction coefficient of 0.6. The marble is dynamically spawned above the plate after the arm reaches its home pose via the `/spawn_entity` Gazebo service. The spawn position is computed at runtime from a TF lookup of the `plate_tcp` frame ($80\,\mathrm{mm}$ above the surface), so no coordinates are hardcoded.

3) State Sensing: Three sensing channels provide the full 8-D state:

- **Marble position** (x, y): The `libgazebo_ros_p3d` Gazebo plugin publishes ground-truth odometry at 50 Hz on `/marble/odom`. Position relative to the plate centre is:

$$x_{\mathrm{rel}} = -(x_{\mathrm{world}} - x_{\mathrm{plate}}), \quad y_{\mathrm{rel}} = y_{\mathrm{world}} - y_{\mathrm{plate}}. \tag{1}$$

  The x axis is negated because `plate_tcp` has yaw $\approx 180°$ at the home configuration, inverting the world-frame X offset relative to the LQR model convention.
- **Marble velocity** $(\dot{x}, \dot{y})$: Numerically differentiated from consecutive world-frame positions and filtered with an exponential moving average ($\tau = 0.08\,\mathrm{s}$).
- **Plate angles and rates** $(\alpha, \dot{\alpha}, \beta, \dot{\beta})$: Roll and pitch are extracted from the `world`→`plate_tcp` TF quaternion. Their time derivatives are computed in the same 50 Hz odometry callback and filtered with the same EMA, guaranteeing sign consistency between angles and rates by construction.

4) Initialization Sequence: The system uses an event-driven launch sequence with no fixed time delays. Gazebo and MoveIt load first; `go_to_pose` homes the arm; on its exit, `marble_spawner` drops the marble and exits, triggering the controller and auxiliary nodes via ROS2 `OnProcessExit` launch event handlers.

B. Classical Controller: Discrete-Time LQR

1) Physical Model: The ball-plate system is modeled as a planar coupled system. Under the small-angle rolling assumption for a solid sphere, ball acceleration along each axis is proportional to plate tilt:

$$\ddot{x} = -C\,\alpha, \qquad \ddot{y} = -C\,\beta, \tag{2}$$

where

$$C = \frac{M_b\,g}{M_b + I_b/r^2} = \tfrac{5}{7}\,g \approx 7.0\,\mathrm{m/s^2}, \tag{3}$$

with marble mass $M_b = 0.05\,\mathrm{kg}$, moment of inertia $I_b = \frac{2}{5}M_b r^2$, radius $r = 0.015\,\mathrm{m}$, and $g = 9.81\,\mathrm{m/s^2}$.

The robot arm's response to angular velocity commands is modeled as a first-order (PT1) lag with time constant $T = 0.35\,\mathrm{s}$, capturing communication latency, controller bandwidth, and joint servo dynamics:

$$\dot{\omega} = \frac{\omega_{\mathrm{cmd}} - \omega}{T}. \tag{4}$$

2) State Vector (8-D): Combining the ball dynamics (2) and the PT1 model (4) for both axes gives the 8-dimensional state:

$$\mathbf{x} = \begin{bmatrix} x & \dot{x} & y & \dot{y} & \alpha & \omega_\alpha & \beta & \omega_\beta \end{bmatrix}^\top, \tag{5}$$

where α (pitch, rotation about world Y) governs x-axis ball motion, β (roll, rotation about world X) governs y-axis ball motion, and ω_α, ω_β are the PT1 states (actual plate angular velocities). The control inputs are velocity commands sent to MoveIt Servo:

$$\mathbf{u} = \begin{bmatrix} \omega_{\alpha,\mathrm{cmd}} & \omega_{\beta,\mathrm{cmd}} \end{bmatrix}^\top. \tag{6}$$

3) Continuous-Time System Matrices: The continuous-time dynamics $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ are:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -C & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & -C & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & -1/T & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & -1/T \end{bmatrix}, \tag{7}$$

$$\mathbf{B} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1/T & 0 \\ 0 & 0 \\ 0 & 1/T \end{bmatrix}. \tag{8}$$

The two axes (x–α and y–β) are fully decoupled, forming two independent 4th-order subsystems embedded in the 8×8 structure. The only non-trivial couplings are $A_{1,4} = -C$ (plate pitch drives ball $\ddot{x}$), $A_{3,6} = -C$ (plate roll drives ball $\ddot{y}$), and the PT1 poles $A_{5,5} = A_{7,7} = -1/T$.

4) Discretization (Zero-Order Hold): The continuous system is discretized at $\Delta t = 1/30\,\mathrm{s}$ using the matrix exponential:

$$\exp\left(\begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \Delta t\right) = \begin{bmatrix} \mathbf{A}_d & \mathbf{B}_d \\ \mathbf{0} & \mathbf{I} \end{bmatrix}, \tag{9}$$

5) LQR Gain Computation: The discrete-time LQR gain $\mathbf{K} \in \mathbb{R}^{2\times 8}$ is computed once at node startup by solving the Discrete Algebraic Riccati Equation (DARE):

$$\mathbf{P} = \texttt{solve\_discrete\_are}(\mathbf{A}_d, \mathbf{B}_d, \mathbf{Q}, \mathbf{R}), \tag{10}$$

$$\mathbf{K} = (\mathbf{R} + \mathbf{B}_d^\top \mathbf{P}\,\mathbf{B}_d)^{-1}\mathbf{B}_d^\top \mathbf{P}\,\mathbf{A}_d. \tag{11}$$

Cost matrices:

$$\mathbf{Q} = \mathrm{diag}(100,\ 100,\ 100,\ 100,\ 5,\ 0.5,\ 5,\ 0.5), \tag{12}$$

$$\mathbf{R} = 5.0\,\mathbf{I}_{2\times 2}. \tag{13}$$

The state ordering is $[x, \dot{x}, y, \dot{y}, \alpha, \omega_\alpha, \beta, \omega_\beta]$. Position weights (indices 0, 2) penalize centering error; velocity weights (indices 1, 3) provide damping against oscillation. Angle and rate weights (indices 4–7) are kept small to allow the plate to tilt aggressively when needed without incurring excessive control effort cost. Both axes carry symmetric weights reflecting the equal importance of x and y stabilization.

6) Online Control Loop (30 Hz): At each control tick the node:

1) Assembles the 8-D state from filtered odometry and TF data.
2) Computes the error $\mathbf{e} = \mathbf{x} - \mathbf{x}_{\mathrm{des}}$ (setpoint defaults to zero; see Section IV).
3) Applies LQR: $\mathbf{u} = -\mathbf{K}\,\mathbf{e}$.
4) Saturates: $\mathbf{u} = \mathrm{clip}(\mathbf{u},\ -45°/\mathrm{s},\ +45°/\mathrm{s})$.
5) Publishes as a `TwistStamped` (frame: `base_link`) to MoveIt Servo via the mux.

MoveIt Servo converts the Cartesian angular velocity command into joint velocity commands at 30 Hz, tracked by the UR5e's `JointTrajectoryController`. Setting the servo command frame to `plate_tcp` ensures that angular commands route through the shoulder and elbow joints (producing plate tilt) rather than the wrist joints.

III. Preliminary Experiment Results

The following describes observed system behavior during initial simulation runs.

Homing and spawning: The arm reliably reaches the home pose (TCP at approximately $[0.233,\ 0.25,\ 0.8]\,\mathrm{m}$ in the world frame) before the marble is dropped. The TF-based spawn calculation correctly places the marble $80\,\mathrm{mm}$ above the plate surface regardless of minor home pose variation. Landing detection — 5 consecutive odometry ticks within $\pm 1\,\mathrm{cm}$ of the plate surface with $|\dot{z}| < 0.50\,\mathrm{m/s}$ — triggers Servo activation reliably without false positives.

Stabilization — baseline LQR: With the default Q/R weights and no trajectory tracking, the LQR controller successfully stabilizes the marble near the plate center. The marble settles from an initial drop within approximately 2 seconds, with residual oscillation on the order of $1\,\mathrm{cm}$ in steady state. Both X and Y axes stabilize, confirming correct state estimation and sign conventions on both channels.

Fell-off recovery: When the marble rolls off the plate edge, the system correctly detects the fall (marble z drops more than $5\,\mathrm{cm}$ below the plate surface), stops Servo, re-homes the arm, and re-spawns the marble is possible. This recovery cycle completes in approximately 8–10 seconds, and is a preparation for future steps.

IV. Extensions

After establishing the baseline LQR stabilizer, three additional components were developed to extend the system's capabilities.

A. Priority Multiplexer

A priority multiplexer node (`mux_controller`) is interposed between the LQR controller and MoveIt Servo. It subscribes to two twist command streams:

- `/marble_servo/delta_twist_cmds` — LQR output (automatic mode).
- `/manual/delta_twist_cmds` — operator commands (manual mode).

Manual commands take priority and override LQR output immediately. The mux automatically reverts to automatic mode after $0.5\,\mathrm{s}$ of inactivity on the manual channel. The active mode is published on `/mux_controller/mode` (`"auto"` or `"manual"`). This design enables safe operator intervention and plate repositioning without restarting the system.

B. Marble Lissajous Trajectory Tracking

The `marble_lissajous_node` publishes time-varying reference positions on `/marble/desired_pos`, driving the LQR to track a Lissajous curve rather than holding the marble at the plate centre. The reference trajectory is:

$$x_d(t) = A\sin(f_a\,\omega_0\,t + \delta), \qquad y_d(t) = A\sin(f_b\,\omega_0\,t), \tag{14}$$

with $\omega_0 = 2\pi/T_0$. Setting $f_a = 1$, $f_b = 2$, and $\delta = \pi/2$ produces a figure-eight. The controller subtracts the desired position from the measured position in the LQR error vector:

$$e_x = x - x_d(t), \qquad e_y = y - y_d(t). \tag{15}$$

The marble successfully tracks the figure-eight setpoint; tracking error grows with increasing setpoint amplitude and frequency, consistent with the linear model's neglect of large-angle nonlinearities.

C. TCP Lissajous with Feedforward Tilt Compensation

The `tcp_lissajous_node` moves the robot TCP along a Lissajous curve in the world XY plane at constant height while the LQR simultaneously balances the marble. At each $30\,\mathrm{Hz}$ tick it analytically computes:

- **TCP linear velocity** (published to `/tcp/lissajous_vel`), injected directly into the `twist.linear.x/y` fields of the `TwistStamped` sent to Servo alongside the LQR angular commands.
- **Feedforward tilt angles** (published to `/tcp/lissajous_ff_tilt`), which pre-compensate the pseudo-force the marble experiences when the TCP accelerates.

When the TCP accelerates with components $(a_x,\, a_y)$ in the world frame, the marble experiences an inertial pseudo-force equivalent to a plate tilt. The compensating feedforward tilt is:

$$\alpha_{\mathrm{ff}} = -\frac{a_x}{g}\cdot k_{\mathrm{ff}}, \qquad \beta_{\mathrm{ff}} = -\frac{a_y}{g}\cdot k_{\mathrm{ff}}, \tag{16}$$

where k_{ff} is a tunable scale factor. The feedforward tilt is injected into the LQR error vector as an offset on the desired plate angle, so the controller drives the plate toward the compensating tilt:

$$e_\alpha = \alpha - \alpha_{\mathrm{ff}}, \qquad e_\beta = \beta - \beta_{\mathrm{ff}}. \tag{17}$$

With $f_b = 2$ (Y axis at twice the base frequency), the TCP acceleration in Y is $4\times$ larger than in X for equal amplitudes, placing a significantly higher disturbance load on the y–β channel.

V. Future Work

A. Kalman Filter for State Estimation

Marble velocity and plate angular velocity are currently estimated by differentiating position measurements and filtering with a simple exponential moving average. A planned improvement replaces this with a **Kalman Filter** or **Extended Kalman Filter** that fuses odometry, joint state, and TF data using the ball-plate PT1 model as the process model. This will provide optimal state estimates under Gaussian noise assumptions, reduce derivative noise without adding excessive phase lag, and improve controller performance.

B. Reinforcement Learning Residual Controller

A Soft Actor-Critic (SAC) residual policy is partially implemented. The training environment (`ball_plate_env.py`) simulates the PT1 ball-plate physics with Coulomb/viscous friction and domain randomization (mass, friction, time constant). A curriculum trains the agent through four stages of increasing residual authority ($\pm 2 \rightarrow \pm 20\,^\circ/\mathrm{s}$) with Lissajous tracking and external perturbations added progressively. The next step is to complete training, evaluate transfer to Gazebo, and tune the LQR + RL blending.

C. Hardware Transfer (Optional)

The simulation is designed to match real UR5e hardware: the PT1 delay model uses a measured actuator time constant, and `libgazebo_ros_p3d` is replaceable with a physical vision system (e.g., overhead camera with blob detection). The next milestone will validate the controller on a physical UR5e arm, assessing whether the PT1 model adequately captures real actuator dynamics and whether the LQR gains transfer without retuning.

D. Quantitative Performance Metrics

Current evaluation is qualitative. Future work will compute RMS centering error, mean time to stabilize from drop, maximum stable Lissajous amplitude, and a comparative analysis of LQR-only versus LQR+RL performance on identical disturbance sequences.